Lab 44 — Log Investigation: Building an Attack Timeline

> **SY0-701 Objective 4.9** — Given a scenario, use data sources to support an investigation.
>
> **Domain 4** — Security Operations (28%) · **Time:** ~20 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: investigate authentication and web server logs to find a brute-force source, the account it compromised, the privilege escalation that followed and the web attacks, then assemble a single chronological timeline.

You'll build: synthetic `auth.log` and `nginx` access-log data, a set of grep/awk findings (top attacker, successful login, sudo escalation, web attack patterns) and a merged, time-ordered incident timeline.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 43 (packet capture) and Lab 41 (incident response) are related; tools installed in step 1.

Background

- Log data is the most common investigation data source. On the exam, know the types: **OS security logs** (Linux `auth.log`/journal, Windows Security log), **application logs** (such as nginx), **endpoint**, **firewall**, **network** and **IPS/IDS** logs, plus **metadata**.
- Linux authentication events live in `/var/log/auth.log` (Debian/Ubuntu) and in the systemd journal. Key phrases: `Failed password`, `Invalid user`, `Accepted password`/`Accepted publickey`, and `sudo: ... COMMAND=`.
- An **nginx access log** records one line per request in the combined format: client IP, time, request line, status code, bytes and user agent. Web attacks show up as suspicious URIs (`../`, `UNION SELECT`, `<script>`) and scanner user agents.
- An investigation correlates several logs into one **timeline**. Timestamps must be normalised (ideally to UTC) so events from different logs line up; this is why time synchronisation (NTP) matters.
- The goal is to answer who, what, when, where and how, and to separate failed attempts (noise) from the one successful chain that matters.

Steps

1. Install the tools and prepare a work area

Everything here uses standard text tools plus `journalctl` for the systemd journal.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y gawk coreutils
mkdir -p /root/investigation && cd /root/investigation
```

2. Generate a synthetic auth.log

This heredoc writes an auth log telling a story: an attacker at 203.0.113.66 brute-forces SSH, finally succeeds as `webadmin`, and that session then escalates to root with sudo. The quiet 198.51.100.4 is a normal admin, included as a control.

```
cat > /root/investigation/auth.log <<'EOF'
Mar 14 01:02:11 web01 sshd[1140]: Failed password for invalid user admin from 203.0.113.66
port 51001 ssh2
Mar 14 01:02:13 web01 sshd[1141]: Failed password for invalid user root from 203.0.113.66
port 51002 ssh2
Mar 14 01:02:15 web01 sshd[1142]: Failed password for invalid user oracle from
203.0.113.66 port 51003 ssh2
Mar 14 01:02:17 web01 sshd[1143]: Failed password for webadmin from 203.0.113.66 port
51004 ssh2
Mar 14 01:02:19 web01 sshd[1144]: Failed password for webadmin from 203.0.113.66 port
51005 ssh2
Mar 14 01:02:21 web01 sshd[1145]: Failed password for webadmin from 203.0.113.66 port
51006 ssh2
Mar 14 01:02:23 web01 sshd[1146]: Failed password for webadmin from 203.0.113.66 port
51007 ssh2
Mar 14 01:02:25 web01 sshd[1147]: Failed password for webadmin from 203.0.113.66 port
51008 ssh2
Mar 14 01:02:40 web01 sshd[1150]: Accepted password for webadmin from 203.0.113.66 port
51020 ssh2
Mar 14 01:02:40 web01 sshd[1150]: pam_unix(sshd:session): session opened for user webadmin
by (uid=0)
Mar 14 01:03:05 web01 sudo:  webadmin : TTY=pts/0 ; PWD=/home/webadmin ; USER=root ;
COMMAND=/usr/bin/cat /etc/shadow
Mar 14 01:03:30 web01 sudo:  webadmin : TTY=pts/0 ; PWD=/home/webadmin ; USER=root ;
COMMAND=/usr/sbin/useradd -m -s /bin/bash support
Mar 14 01:03:31 web01 sudo:  webadmin : TTY=pts/0 ; PWD=/home/webadmin ; USER=root ;
COMMAND=/usr/bin/passwd support
Mar 14 08:15:02 web01 sshd[2201]: Accepted publickey for ops from 198.51.100.4 port 44512
ssh2
Mar 14 08:16:40 web01 sudo:  ops : TTY=pts/1 ; PWD=/home/ops ; USER=root ;
COMMAND=/usr/bin/apt-get update
EOF
wc -l /root/investigation/auth.log
```

3. Generate a synthetic nginx access log

The same attacker IP probes the web app before the SSH break-in: directory traversal, a SQL injection attempt and a reflected XSS test, with a scanner user agent. Legitimate traffic is mixed in as noise.

```
cat > /root/investigation/access.log <<'EOF'
198.51.100.9 - - [14/Mar/2026:00:58:01 +0000] "GET / HTTP/1.1" 200 1024 "-" "Mozilla/5.0"
203.0.113.66 - - [14/Mar/2026:00:59:11 +0000] "GET /index.php?page=../../../../etc/passwd
HTTP/1.1" 200 2048 "-" "sqlmap/1.7"
203.0.113.66 - - [14/Mar/2026:00:59:14 +0000] "GET /products?
id=1%20UNION%20SELECT%20username,password%20FROM%20users HTTP/1.1" 500 512 "-"
"sqlmap/1.7"
203.0.113.66 - - [14/Mar/2026:00:59:18 +0000] "GET /search?q=<script>alert(1)</script>
HTTP/1.1" 200 640 "-" "sqlmap/1.7"
203.0.113.66 - - [14/Mar/2026:00:59:40 +0000] "POST /admin/login HTTP/1.1" 401 180 "-"
"sqlmap/1.7"
203.0.113.66 - - [14/Mar/2026:01:00:05 +0000] "POST /admin/login HTTP/1.1" 401 180 "-"
```

```
"sqlmap/1.7"
203.0.113.66 - - [14/Mar/2026:01:00:22 +0000] "POST /admin/login HTTP/1.1" 302 0 "-"
"sqlmap/1.7"
198.51.100.9 - - [14/Mar/2026:01:05:00 +0000] "GET /about HTTP/1.1" 200 980 "-"
"Mozilla/5.0"
203.0.113.66 - - [14/Mar/2026:01:06:10 +0000] "GET /uploads/shell.php?cmd=id HTTP/1.1" 200
44 "-" "curl/8.0"
EOF
wc -l /root/investigation/access.log
```

4. Find the brute-force source and count the failures

Start with the authentication log. Count failed passwords per source IP to find the loudest attacker.

```
cd /root/investigation
grep 'Failed password' auth.log | grep -oE 'from [0-9.]+' | awk '{print $2}' | sort | uniq
-c | sort -rn
echo "--- usernames the attacker tried ---"
grep 'Failed password' auth.log | grep -oE '(invalid user [a-z]+|for [a-z]+) from' | sed -
E 's/ from//; s/invalid user //; s/for //' | sort | uniq -c | sort -rn
```

```
      8 203.0.113.66
--- usernames the attacker tried ---
      5 webadmin
      1 root
      1 oracle
      1 admin
```

5. Find the successful login and link it to the brute force

A successful login from the same IP that was just failing, especially right after a run of failures, is the moment the brute force worked.

```
grep 'Accepted' auth.log
echo "--- did the successful login come from the brute-force IP? ---"
grep -E 'Accepted .* from 203.0.113.66' auth.log && echo ">> webadmin was compromised from
the attacker IP"
```

```
Mar 14 01:02:40 web01 sshd[1150]: Accepted password for webadmin from 203.0.113.66 port
51020 ssh2
Mar 14 08:15:02 web01 sshd[2201]: Accepted publickey for ops from 198.51.100.4 port 44512
ssh2
>> webadmin was compromised from the attacker IP
```

6. Find the privilege escalation

After the compromise, follow what the account did with sudo. Reading `/etc/shadow` and creating a new account are classic post-exploitation actions.

```
grep 'sudo:' auth.log | sed -E 's/.*sudo:  *([a-z]+) .*USER=([a-z]+) ; COMMAND=(.*)/\1 ->
\2 : \3/'
```

```
echo "--- persistence: new account created by the compromised user ---"
grep -E 'sudo:.*webadmin.*useradd' auth.log
```

```
webadmin -> root : /usr/bin/cat /etc/shadow
webadmin -> root : /usr/sbin/useradd -m -s /bin/bash support
webadmin -> root : /usr/bin/passwd support
ops -> root : /usr/bin/apt-get update
```

7. Investigate the web attacks

Now turn to the application log. Find scanner user agents, web attack signatures and the response codes that reveal which attempts worked (a 302 after many 401s is a successful login; a 200 on a web shell is bad news).

```
cd /root/investigation
echo "--- requests from the attacker IP ---"
awk '$1=="203.0.113.66"' access.log | awk -F'"' '{print $2, "=>", $3}'
echo "--- attack signatures ---"
grep -En '\.\./|UNION SELECT|<script>|shell\.php' access.log
echo "--- auth attempts on /admin/login by status ---"
awk '/\/admin\/login/ {print $9}' access.log | sort | uniq -c
```

```
--- attack signatures ---
2:...etc/passwd...        (directory traversal)
3:...UNION SELECT...      (SQL injection)
4:...<script>alert(1)...  (reflected XSS)
9:...shell.php?cmd=id...  (web shell)
--- auth attempts on /admin/login by status ---
   2 401
   1 302
```

8. Use journalctl for the same questions on a live system

On a running host the events are in the systemd journal, not always a flat file. The same investigation uses journal filters. Seed one event so the commands return data in the playground.

```
logger -p auth.info -t sshd "Accepted password for webadmin from 203.0.113.66 port 51020
ssh2"
journalctl -t sshd --since "-5 min" --no-pager | tail -3
journalctl _COMM=sudo --since today --no-pager | tail -3 2>/dev/null || echo "(no sudo
events in the journal yet)"
journalctl -t sshd --since "-10 min" -o short-iso --no-pager | grep -c 'Accepted\|Failed'
|| true
```

9. Build the merged timeline

Correlate both logs into one chronological timeline. Convert each log's timestamp to a common `HH:MM:SS` key, tag the source, and sort. This is the single artefact that tells the whole story.

```
cd /root/investigation
{
  awk '{t=$1" "$2" "$3; sub(/:[0-9]+$/,"",$0)}
```

```
      /Failed password/      {print $3"  AUTH  brute-force failure from "$0; }' auth.log
>/dev/null
  # auth.log: field 3 is HH:MM:SS
  grep -E 'Failed password|Accepted|sudo:' auth.log | awk '{printf "%s  AUTH   %s\n", $3,
substr($0, index($0,$5))}'
  # access.log: timestamp is inside [dd/Mon/yyyy:HH:MM:SS +0000]
  awk -F'[][]' '{split($2,a,":"); printf "%s:%s:%s  WEB    %s %s %s\n", a[2],a[3],a[4],
$1, $3, ""}' access.log \
    | sed -E 's/(WEB  +[0-9.]+) - - /\1 /'
} | sort > timeline.txt
sed -n '1,30p' timeline.txt
```

```
00:58:01  WEB    198.51.100.9 "GET / HTTP/1.1" 200
00:59:11  WEB    203.0.113.66 "GET /index.php?page=../../../../etc/passwd HTTP/1.1" 200
...
01:02:17  AUTH   Failed password for webadmin from 203.0.113.66 port 51004 ssh2
01:02:40  AUTH   Accepted password for webadmin from 203.0.113.66 port 51020 ssh2
01:03:05  AUTH   sudo:  webadmin : ... COMMAND=/usr/bin/cat /etc/shadow
```

Test it

```
cd /root/investigation
echo "Top attacker: $(grep 'Failed password' auth.log | grep -oE 'from [0-9.]+' | awk
'{print $2}' | sort | uniq -c | sort -rn | head -1)"
grep -q 'Accepted password for webadmin from 203.0.113.66' auth.log && echo "PASS:
compromised account identified"
grep -qE 'useradd .* support' auth.log && echo "PASS: persistence (new account) found"
test -s timeline.txt && echo "PASS: timeline built with $(wc -l < timeline.txt) events"
```

Expected: the top attacker is `203.0.113.66` with 8 failures, the compromise of `webadmin` and the `support` account creation are both found, and the timeline contains all the merged events in time order.

Troubleshooting

Symptom	Fix
`journalctl` returns nothing in step 8	A minimal playground may not persist the journal. The flat-file analysis in steps 2 to 7 is the authoritative version; step 8 just shows the live-system equivalent.
The timeline mixes the orders of events at the same second	Sub-second order needs millisecond timestamps. Note the limitation; in production, enable high-precision timestamps and synchronise clocks with NTP.
awk prints blank WEB lines	The access log must be in the exact combined format from step 3. Re-run the heredoc; a wrapped or edited line breaks the `[...]` field split.

Challenge

Extend the investigation: write a one-line report that states the attacker IP, the compromised account, the UTC time of compromise, the privilege escalation commands and the indicator you would block first. Then add an nginx log line showing the attacker downloading a file through the web shell and show how it changes your timeline.

Reflection

You found 8 failed logins and 1 success from the same IP. Why is correlating the web log with the auth log more convincing than either log alone, and what does the quiet 198.51.100.4 teach you about false positives?

Exam tips

- Know the log sources by name: OS-specific security logs (auth.log, Windows Security), application logs (nginx), firewall, network, IPS/IDS, and metadata.
- A successful login immediately after many failures from the same source is the signature of a successful brute force or password spray.
- Correlation across sources builds the timeline; synchronised clocks (NTP) and consistent time zones (UTC) make correlation possible.
- `sudo ... COMMAND=` lines are your record of privilege escalation and what the attacker did as root.

References

- NIST SP 800-92 Guide to Computer Security Log Management — https://csrc.nist.gov/pubs/sp/800/92/final
- journalctl(1) manual — https://www.freedesktop.org/software/systemd/man/journalctl.html
- OWASP Top 10 (injection, XSS) — https://owasp.org/www-project-top-ten/
- MITRE ATT&CK T1110 Brute Force — https://attack.mitre.org/techniques/T1110/
- SANS log analysis resources — https://www.sans.org/blog/
